

EXPERIMENT NO. 9

AIM: To implement Local Storage and Session Storage in a web application to save and manage user preferences such as theme settings using HTML, CSS, and JavaScript.

TOOLS / SOFTWARE REQUIRED

- Visual Studio Code (VS Code)
- Google Chrome
- HTML5
- CSS3
- JavaScript (ES6)
- Web Browser with Local Storage and Session Storage support

THEORY

1. Web Storage in JavaScript

Web Storage is a browser-based mechanism that allows web applications to store small amounts of data as key-value pairs. JavaScript provides two main Web Storage objects: `localStorage` and `sessionStorage`. They are useful for saving user preferences and temporary session information without requiring a server-side database.

In this experiment, Web Storage is used to store the user's selected theme. The selected theme can be restored when the webpage is opened again, providing a more convenient and personalized user experience.

2. Local Storage

`localStorage` is used to store data in the user's browser so that the data remains available even after the webpage is closed or the browser is restarted. Data is stored as key-value pairs and remains until it is explicitly removed by the application or the user.

In this practical, the selected theme is stored using the key "theme".

Example:

```
localStorage.setItem("theme", theme);
```

The above statement stores the selected theme in local storage. The stored value can later be retrieved using `getItem()`.

3. Session Storage

`sessionStorage` is similar to `localStorage`, but its data is intended for the current browser session. It is useful when information is required temporarily while the user is using the webpage.

In this experiment, the current theme is also stored using `sessionStorage` with the key "currentTheme".

Example:

```
sessionStorage.setItem("currentTheme", theme);
```

This stores the current theme for the active browser session.

4. Difference Between Local Storage and Session Storage

Both `localStorage` and `sessionStorage` store data as key-value pairs, but their lifetime is different.

- localStorage keeps the stored preference until it is explicitly removed.
- sessionStorage keeps data for the current browser session.
- localStorage is suitable for persistent preferences such as a preferred theme.
- sessionStorage is suitable for temporary information required during a session.
- Both can be accessed through JavaScript using similar methods such as setItem(), getItem(), removeItem(), and clear().

5. Storing User Preference

A user preference is a setting selected by the user that can be remembered by a web application. Examples include theme selection, language preference, layout preference, and display settings.

In this practical, the user can select either a Light Theme or a Dark Theme. JavaScript applies the selected theme to the webpage and saves the preference in localStorage.

6. setItem() Method

The setItem() method is used to store a value in Web Storage using a specified key.

Example:

```
localStorage.setItem("theme", theme);
```

Here, "theme" is the key and the selected theme is the value. The same method is used with sessionStorage to store the current session preference.

7. getItem() Method

The getItem() method retrieves a previously stored value from Web Storage.

Example:

```
let savedTheme = localStorage.getItem("theme");
```

When the webpage loads, JavaScript checks whether a theme has already been saved. If a value exists, the saved theme is applied automatically.

8. removeItem() and Clearing Preferences

The removeItem() method removes a particular key-value pair from storage. In this practical, it is used to remove the saved theme preference.

Example:

```
localStorage.removeItem("theme");
```

The practical also removes the temporary session value using sessionStorage.removeItem("currentTheme"). After clearing the preference, the webpage returns to the default light appearance.

9. DOM Manipulation and Theme Application

The Document Object Model (DOM) allows JavaScript to access and modify HTML elements dynamically. In this experiment, the body element is modified by adding or removing the CSS class "dark".

For the dark theme, JavaScript uses:

```
document.body.classList.add("dark");
```

For the light theme, the class is removed using:

```
document.body.classList.remove("dark");
```

This demonstrates how JavaScript can dynamically change the appearance of a webpage by manipulating CSS classes.

10. Event Handling and Page Loading

JavaScript event handling is used to respond to user actions. In this practical, button click events are handled using the onclick attribute. When the user clicks the Light Theme or Dark Theme button, the setTheme() function is called.

The clearTheme() function is executed when the Clear Preference button is clicked.

The window.onload event is used when the webpage opens. It checks localStorage for a previously saved theme and automatically applies it if a value is available.

Example:

```
window.onload = function () {  
    let savedTheme = localStorage.getItem("theme");  
    if (savedTheme) {  
        setTheme(savedTheme);  
    }  
};
```

11. Dynamic User Feedback

The application provides feedback to the user through the message paragraph. After a theme is selected, the message displays the saved theme. When the preference is cleared, the message informs the user that the theme preference has been cleared.

Example:

```
document.getElementById("message").innerText =  
    "Theme saved: " + theme;
```

This makes the application more interactive and clearly communicates the result of each user action.

12. Working of the Practical

The working of the application can be summarized as follows:

1. The webpage displays Light Theme, Dark Theme, and Clear Preference buttons.
2. When the user selects a theme, JavaScript applies the corresponding CSS class.
3. The selected theme is saved in localStorage using the key "theme".
4. The current theme is also stored in sessionStorage using the key "currentTheme".
5. When the webpage is opened again, window.onload retrieves the saved theme from localStorage.
6. The saved theme is automatically applied to the webpage.
7. When Clear Preference is clicked, the stored theme values are removed and the webpage returns to the light theme.
8. A message is displayed to provide feedback to the user.

CONCLUSION

The experiment was successfully performed to implement Local Storage and Session Storage using HTML, CSS, and JavaScript. The application stores the user's theme preference, restores the saved theme when the webpage is opened, and provides an option to clear the stored preference. DOM manipulation, CSS class handling, JavaScript event handling, localStorage, sessionStorage, and dynamic user feedback were implemented. The practical demonstrated how Web Storage can be used to create a personalized and user-friendly web application without requiring a server-side database.

RESULT

The web application successfully saved, retrieved, and cleared the user's theme preference using Local Storage and Session Storage.